



**KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN MẠNG VÀ CÁC HỆ THỐNG THÔNG TIN**

CHƯƠNG 2

Cấu trúc Hệ điều hành



Nội dung chính - Chương 2

Mục lục

- 2.1. Giới thiệu
- 2.2. Các thành phần hệ thống
- 2.3. Các dịch vụ hệ điều hành
- 2.4. Lời gọi hệ thống
- 2.5. Các chương trình hệ thống
- 2.6. Cấu trúc hệ thống

Từ khóa

Kernel	System Service
Process	Memory
File System	System Call
API	System Program
Microkernel	Module

Trọng tâm kiến thức



Nhận diện thành phần

Mô tả vai trò của quản lý tiến trình, bộ nhớ, tệp, I/O và bảo vệ.

Hiểu giao diện hệ thống

Giải thích dịch vụ, API, lời gọi hệ thống và chương trình hệ thống.

So sánh cấu trúc

Phân biệt đơn khối, phân tầng, vi nhân, mô-đun và cấu trúc lai.

Câu hỏi định hướng: Hệ điều hành tổ chức các chức năng bên trong như thế nào để vừa hiệu quả, vừa dễ mở rộng và an toàn?



2.1. Giới thiệu

Nhìn hệ điều hành từ dịch vụ, giao diện và cấu trúc bên trong



Ba góc nhìn về cấu trúc hệ điều hành

Người dùng

Quan tâm hệ thống thuận tiện, ổn định, an toàn và phản hồi tốt.

Lập trình viên

Cần API nhất quán và các dịch vụ che giấu chi tiết phần cứng.

Người thiết kế

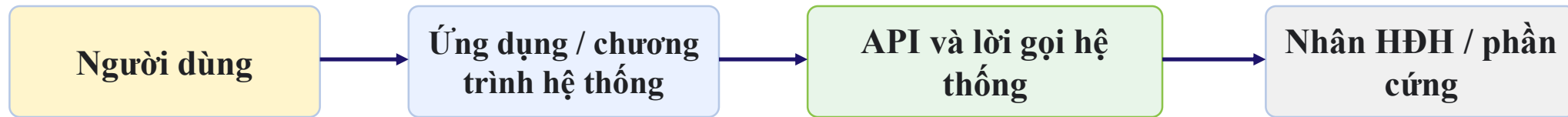
Tổ chức thành phần, giao diện và cơ chế phối hợp bên trong.

Cấu trúc HĐH trả lời ba câu hỏi: Hệ thống cung cấp gì, chương trình truy cập bằng cách nào, và các phần bên trong phối hợp ra sao?



Từ yêu cầu của người dùng đến thao tác phần cứng

Mỗi lớp cung cấp một mức trừu tượng cao hơn lớp bên dưới.



Ứng dụng không cần biết chi tiết thiết bị; nhân HĐH kiểm tra quyền, quản lý tài nguyên và thực hiện yêu cầu một cách có kiểm soát.



2.2. Các thành phần hệ thống

Các khối chức năng chịu trách nhiệm quản lý tài nguyên và bảo đảm hoạt động đúng đắn

Bản đồ các thành phần của hệ điều hành



Quản lý tiến trình



Đơn vị công việc

Tiến trình là chương trình đang thực thi và mang trạng thái riêng.

Tài nguyên

Cần thời gian CPU, bộ nhớ, tệp và thiết bị vào/ra để hoàn thành tác vụ.

Điều phối

HDH lập lịch, đồng bộ và kiểm soát tương tác giữa các tiến trình.

Khi tiến trình kết thúc, hệ điều hành thu hồi các tài nguyên có thể tái sử dụng.



Nhiệm vụ của quản lý tiến trình

Quản lý vòng đời

Tạo và xóa tiến trình

Tạm dừng và tiếp tục

Đồng bộ và giao tiếp

Phòng ngừa / xử lý bế tắc

Phối hợp đồng thời



Quản lý bộ nhớ

Theo dõi

Biết vùng nhớ nào đang dùng và tiến trình nào đang sở hữu.

Quyết định

Chọn tiến trình và phần dữ liệu cần nạp khi bộ nhớ sẵn sàng.

Cấp phát

Cấp phát, bảo vệ và thu hồi không gian bộ nhớ theo yêu cầu.

Quản lý bộ nhớ tạo nền tảng cho đa chương trình, cách ly tiến trình và bộ nhớ ảo.



Quản lý hệ thống tệp và lưu trữ

Tệp và thư mục

Tạo, xóa, đọc, ghi, tìm kiếm và liệt kê thông tin.

Ảnh xạ lưu trữ

Gắn cấu trúc logic của tệp với khối dữ liệu trên thiết bị.

Bền vững

Quản lý không gian trống, hiệu năng, sao lưu và khôi phục.

HĐH cung cấp một không gian tên thống nhất dù dữ liệu nằm trên nhiều loại phương tiện vật lý.



Quản lý vào/ra và thiết bị

Trừu tượng hóa I/O

Cung cấp giao diện chung để chương trình đọc, ghi và điều khiển thiết bị mà không phụ thuộc chi tiết phần cứng.

Trình điều khiển thiết bị

Chuyển yêu cầu chung thành lệnh đặc thù; xử lý ngắt, bộ đệm, cache và hàng đợi I/O.

Mục tiêu: Che giấu sự đa dạng thiết bị, tăng hiệu năng và ngăn chương trình truy cập phần cứng tùy tiện.



Bảo vệ, bảo mật và dịch vụ mạng

Bảo vệ nội bộ

Kiểm soát quyền truy cập CPU, vùng nhớ, tệp và thiết bị giữa người dùng hoặc tiến trình; bảo đảm cách ly.

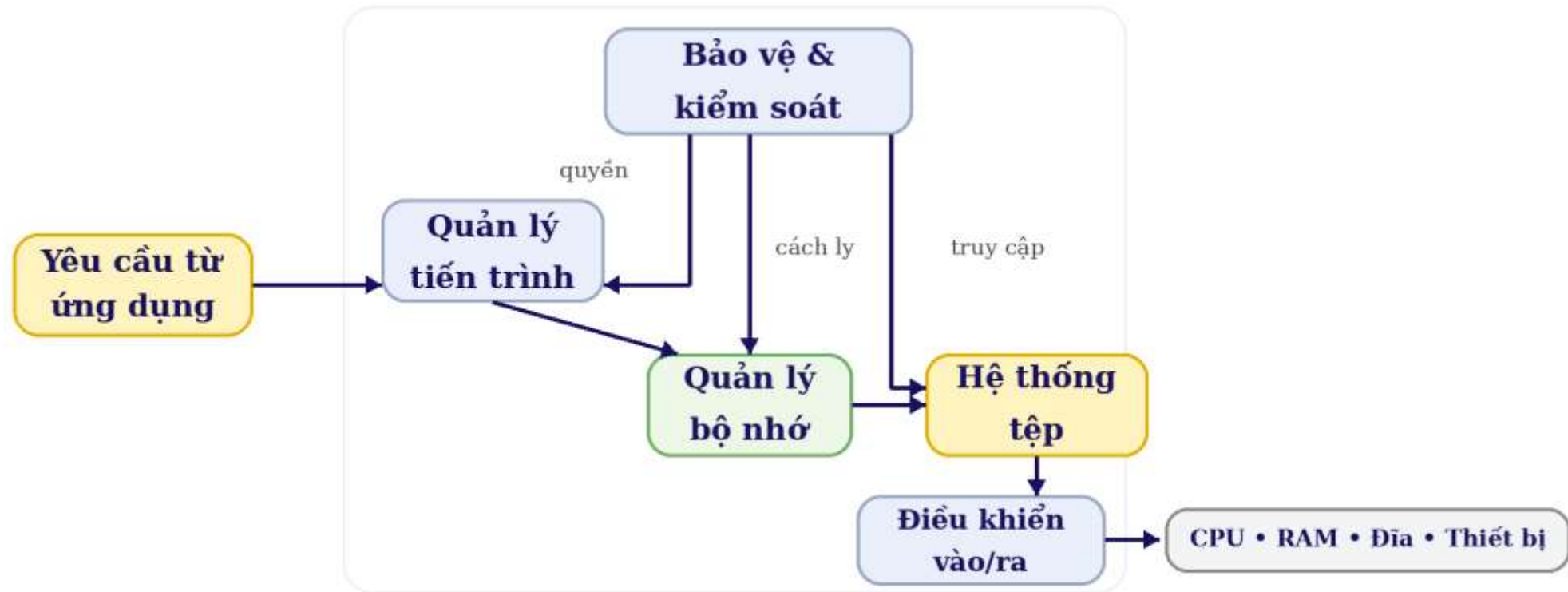
Bảo mật và mạng

Xác thực, phân quyền, ghi vết, bảo vệ giao tiếp và cung cấp cơ chế truyền thông qua mạng.

Bảo vệ quy định ai được làm gì; bảo mật chống truy cập hoặc phá hoại trái phép từ bên trong lẫn bên ngoài.



Các thành phần phối hợp để xử lý một yêu cầu



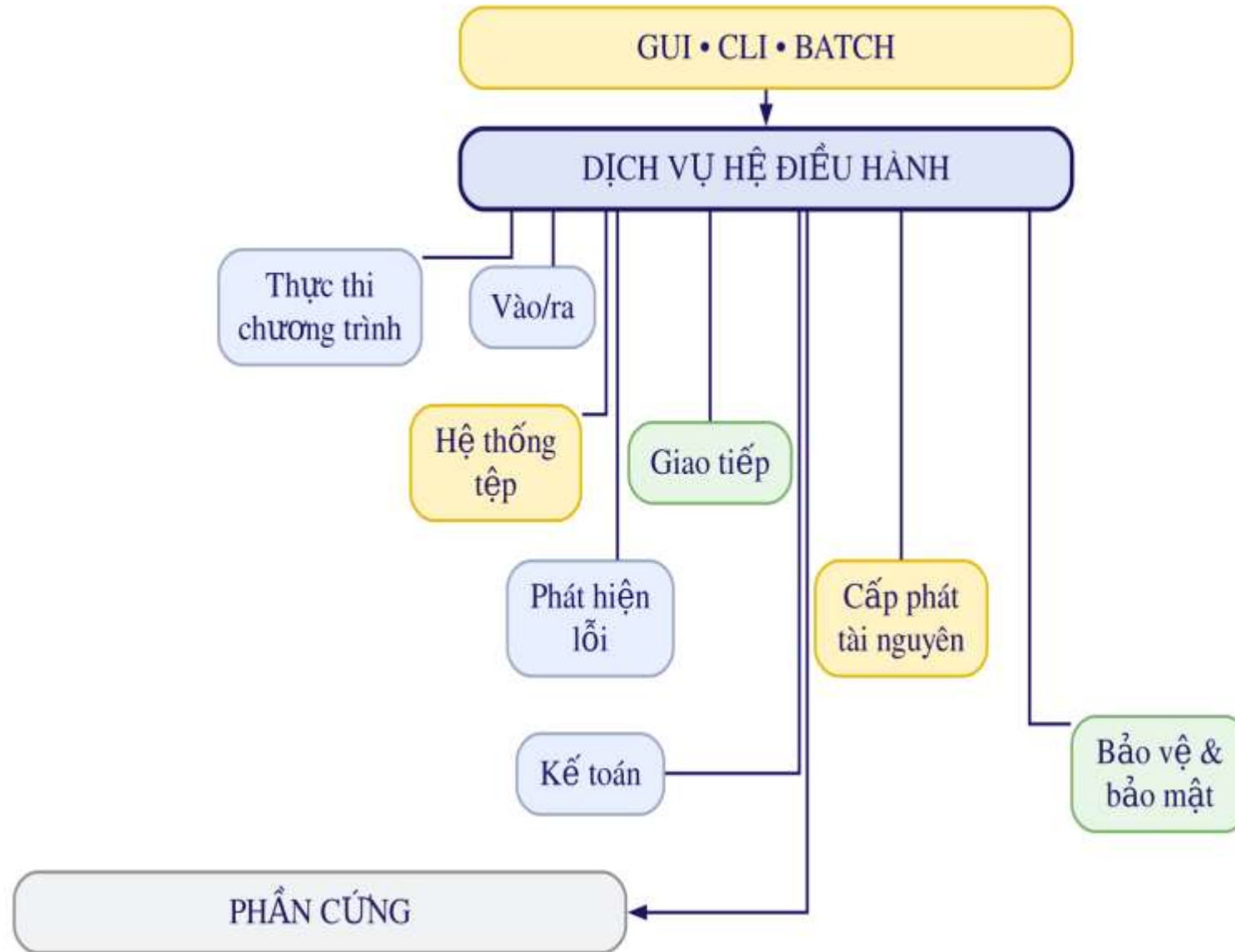
Một thao tác đơn giản có thể đi qua quản lý tiến trình, bộ nhớ, tệp, I/O và cơ chế bảo vệ trước khi tới phần cứng.



2.3. Các dịch vụ hệ điều hành

Những chức năng HĐH cung cấp cho người dùng, chương trình và chính hệ thống

Bản đồ các dịch vụ hệ điều hành





Nhóm dịch vụ hướng đến người dùng

Giao diện người dùng

CLI, GUI hoặc Batch giúp nhập lệnh và điều khiển công việc.

Thực thi chương trình

Nạp, chạy và kết thúc chương trình bình thường hoặc khi có lỗi.

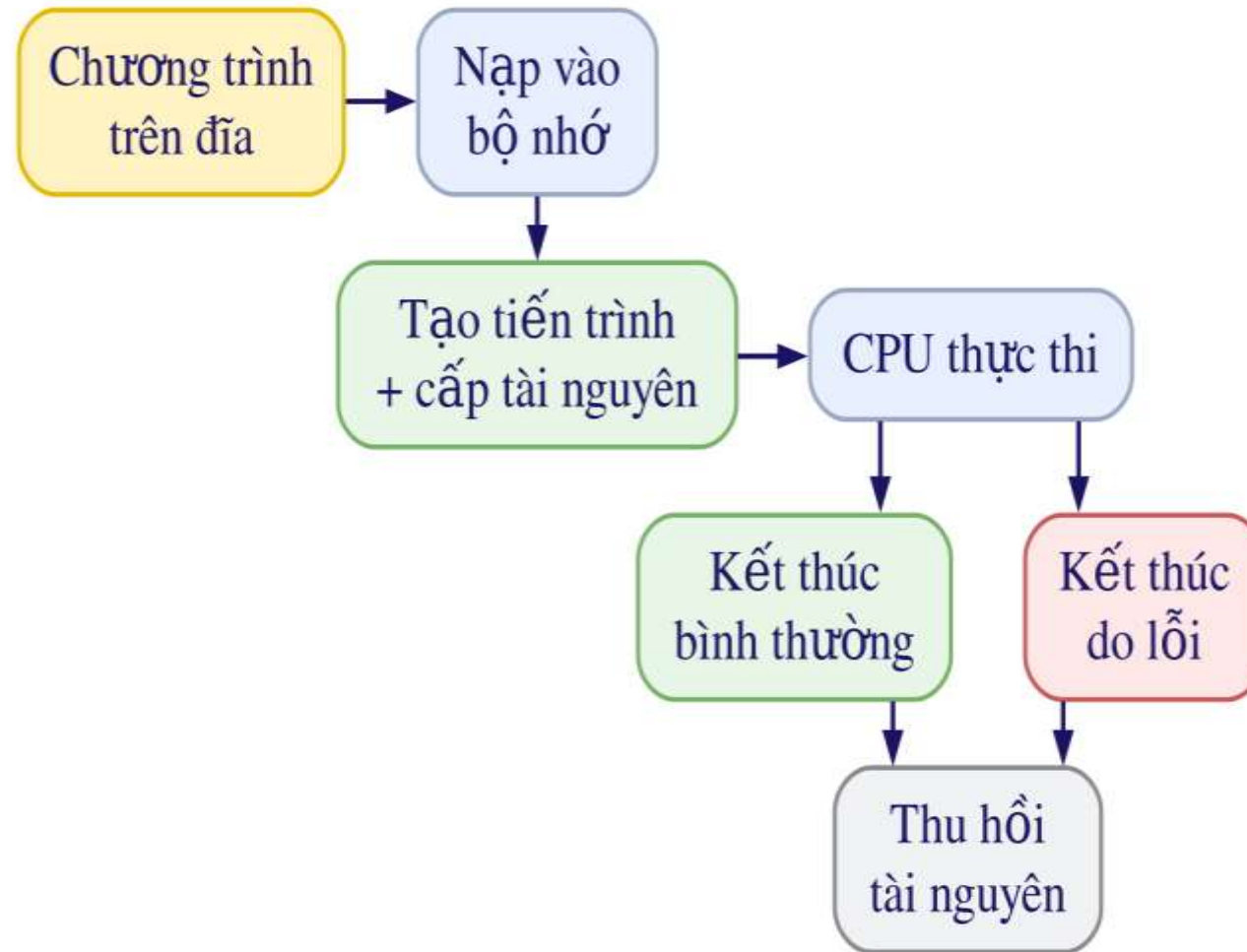
Tệp và vào/ra

Đọc, ghi, tạo, xóa và thao tác trên tệp, thư mục, thiết bị.

Các dịch vụ này làm giảm khối lượng công việc của người lập trình và tạo môi trường thực thi thuận tiện.



Dịch vụ thực thi chương trình quản lý toàn bộ vòng đời





Dịch vụ vào/ra và hệ thống tệp

Thao tác vào/ra

Đọc, ghi và điều khiển thiết bị thông qua giao diện chuẩn; HĐH kiểm soát quyền và thứ tự truy cập.

Thao tác hệ thống tệp

Tạo, mở, đóng, đọc, ghi, đổi tên, xóa, tìm kiếm và quản lý quyền của tệp/thư mục.

Người dùng thấy tệp và luồng dữ liệu; HĐH xử lý khối đĩa, bộ đệm, ngắt và trình điều khiển ở phía dưới.



Giao tiếp và phát hiện lỗi

Giao tiếp

Tiến trình trao đổi dữ liệu bằng bộ nhớ chia sẻ hoặc truyền thông báo, trên cùng máy hoặc qua mạng.

Phát hiện và xử lý lỗi

Theo dõi lỗi CPU, bộ nhớ, I/O và chương trình; trả mã lỗi, kết thúc tiến trình hoặc dừng hệ thống khi cần.

Mục tiêu chung là duy trì tính đúng đắn và nhất quán của quá trình tính toán.



Nhóm dịch vụ giúp hệ thống vận hành hiệu quả

Cấp phát tài nguyên

Phân phối CPU, bộ nhớ, lưu trữ và thiết bị cho các công việc cạnh tranh.

Kế toán

Theo dõi mức sử dụng tài nguyên để thống kê, tính phí hoặc tối ưu hệ thống.

Bảo vệ và bảo mật

Kiểm soát truy cập, xác thực và ngăn tiến trình can thiệp lẫn nhau.

Dịch vụ hướng hệ thống ưu tiên hiệu quả, công bằng, khả năng kiểm soát và độ tin cậy.

Ba kiểu giao diện người dùng phổ biến

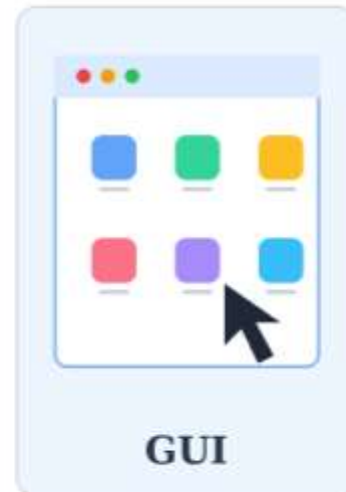
CLI

Lệnh văn bản; nhanh, có thể tạo script và phù hợp quản trị hệ thống.



GUI

Cửa sổ, biểu tượng, menu; trực quan và thuận tiện cho người dùng phổ thông.



Batch

Chuỗi lệnh được chuẩn bị trước và thực hiện không cần tương tác liên tục.



Một hệ điều hành có thể cung cấp đồng thời nhiều giao diện; lựa chọn phụ thuộc nhiệm vụ và nhóm người dùng.



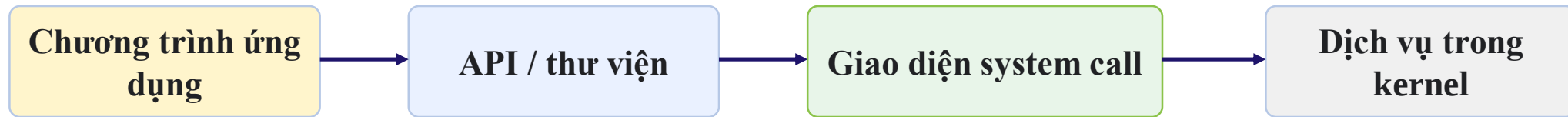
2.4. Lời gọi hệ thống

Giao diện lập trình cho phép chương trình yêu cầu dịch vụ của nhân hệ điều hành



Lời gọi hệ thống là cổng vào có kiểm soát của nhân

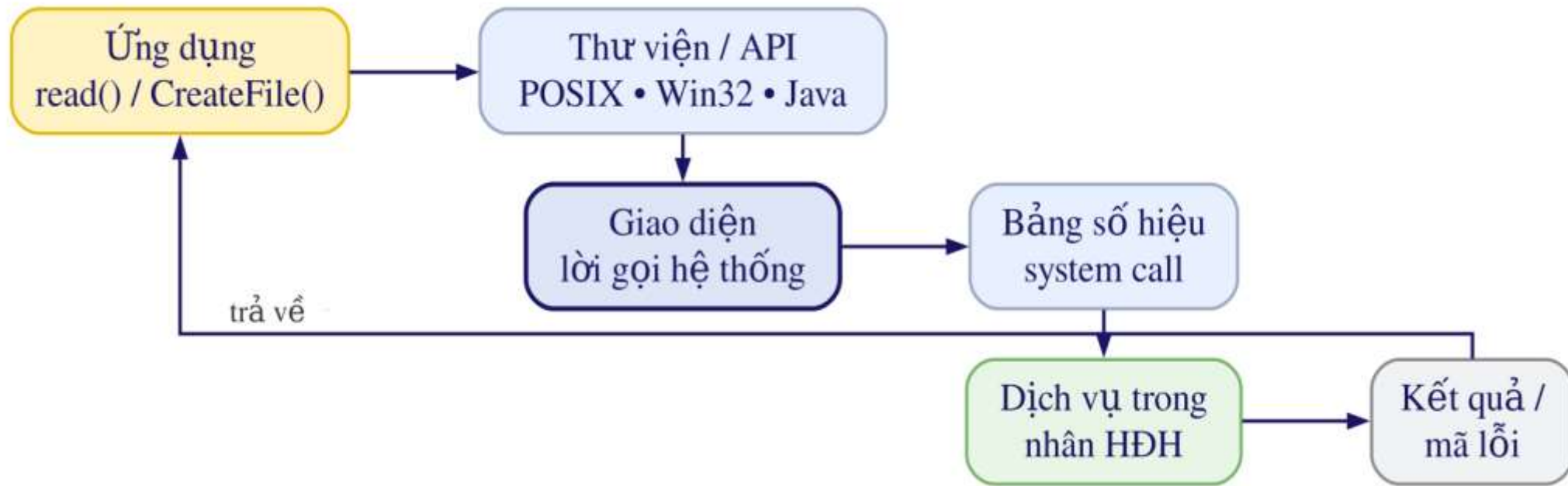
Ứng dụng thường gọi hàm thư viện hoặc API thay vì thao tác trực tiếp với nhân.



Mỗi lời gọi thường gắn với một số hiệu; giao diện system call tra bảng, chuyển điều khiển đến thủ tục tương ứng rồi trả trạng thái và kết quả.



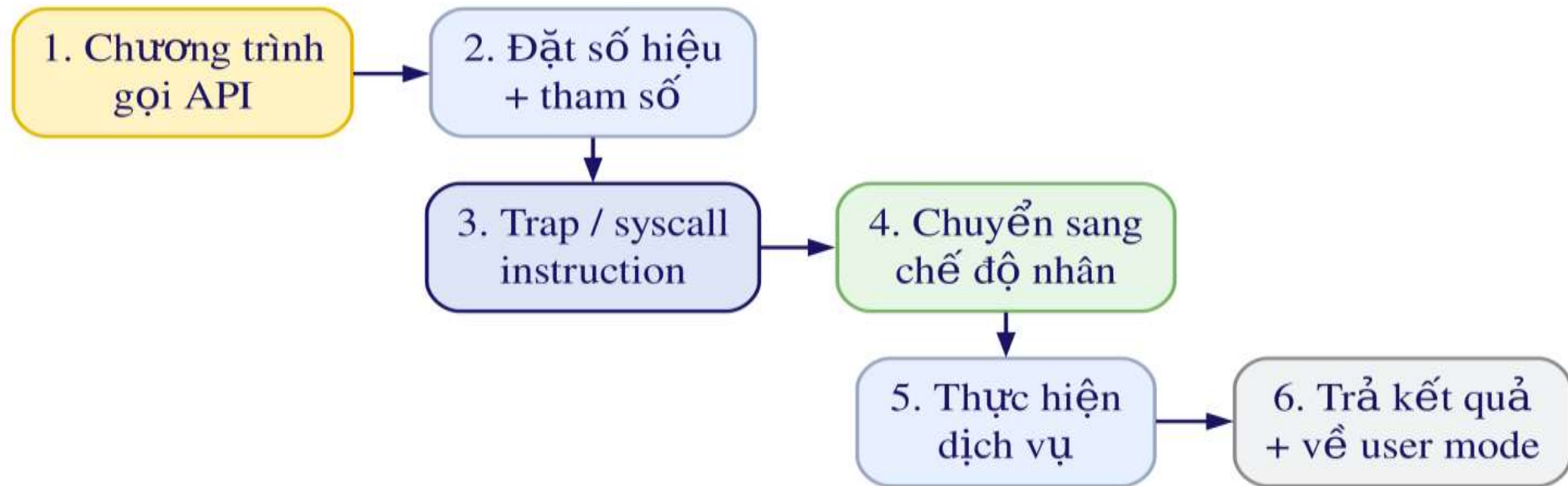
Quan hệ giữa API, lời gọi hệ thống và nhân HĐH



API tạo giao diện thuận tiện và khả chuyển; system call là cơ chế chuyển yêu cầu vào nhân để thực hiện dịch vụ đặc quyền.

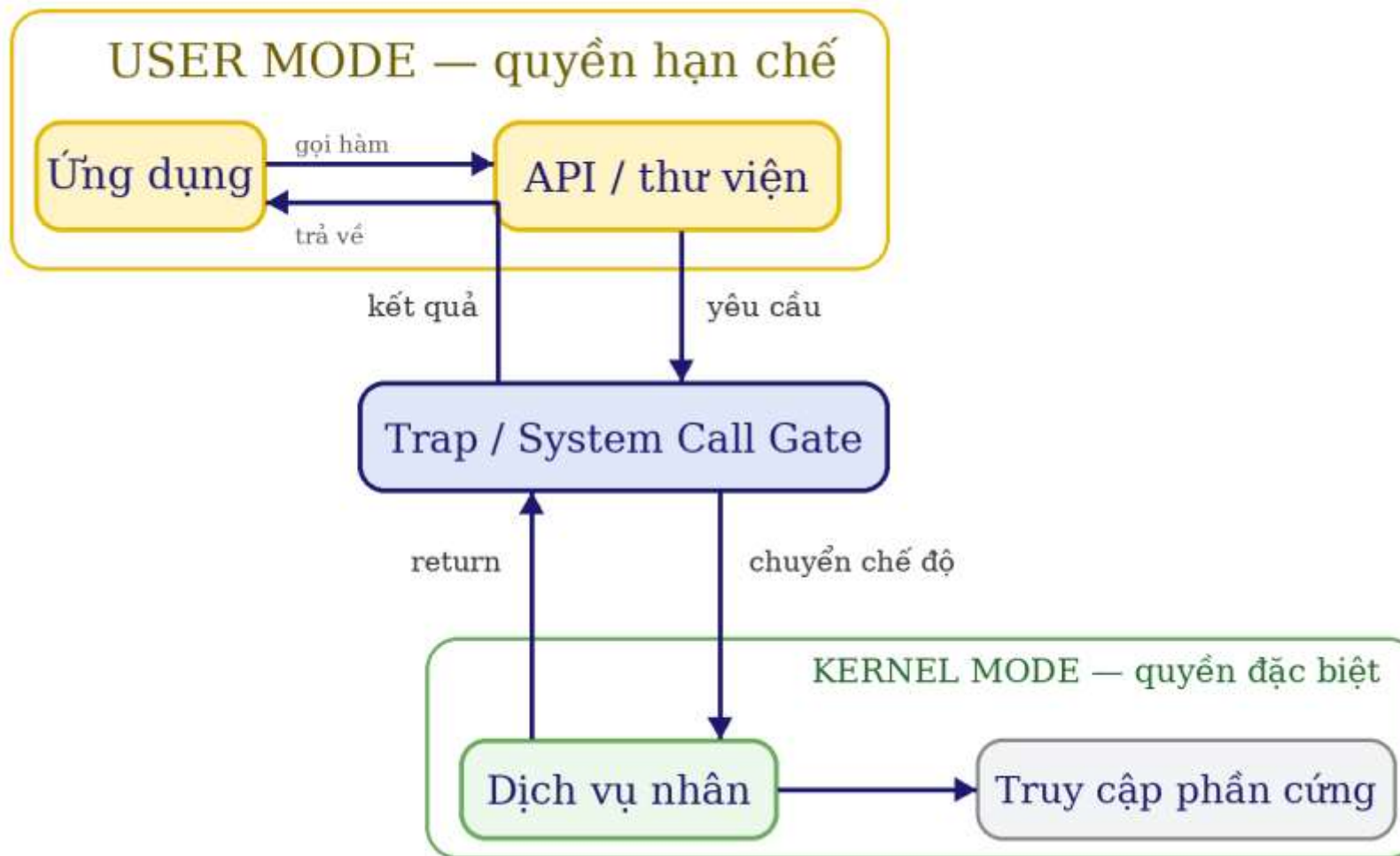


Luồng thực hiện một lời gọi hệ thống



Lệnh trap/syscall chuyển CPU sang chế độ nhân; sau khi dịch vụ hoàn tất, kết quả và mã lỗi được trả về chương trình.

Chuyển đổi giữa chế độ người dùng và chế độ nhân





Ba phương pháp truyền tham số cho lời gọi hệ thống

Thanh ghi

Nhanh và đơn giản, nhưng bị giới hạn bởi số lượng và kích thước thanh ghi.

Khối trong bộ nhớ

Chương trình đặt tham số vào bảng/khối rồi truyền địa chỉ qua thanh ghi.

Ngăn xếp

Chương trình đẩy tham số vào stack; nhân lấy ra theo quy ước đã xác định.

Khối nhớ và ngăn xếp hỗ trợ nhiều tham số hơn; hệ điều hành phải kiểm tra địa chỉ và dữ liệu đến từ không gian người dùng.



Các nhóm lời gọi hệ thống

Nhóm chức năng

1. Quản lý tiến trình
2. Quản lý tệp
3. Quản lý thiết bị
4. Duy trì thông tin
5. Giao tiếp
6. Bảo vệ

Ví dụ

1 · fork / exec

1 · exit / wait

2 · open / close

2 · read / write

3 · ioctl

4 · getpid

5 · pipe / socket

5 · mmap

6 · chmod

6 · setuid



Lời gọi quản lý tiến trình

Tạo và kết thúc

Tạo tiến trình mới, nạp chương trình, kết thúc tiến trình và chờ tiến trình con: `fork()`, `exec()`, `exit()`, `wait()`.

Điều khiển thực thi

Nạp/thi hành, lấy hoặc đặt thuộc tính, cấp phát bộ nhớ, gửi tín hiệu và quản lý trạng thái tiến trình.

Lời gọi quản lý tiến trình tạo cơ chế để hình thành, điều khiển và phối hợp các đơn vị công việc trong hệ thống.

Quản lý tệp và thiết bị



Tệp và thư mục

create, open, read, write, reposition, close, delete; lấy/đặt thuộc tính và quyền truy cập.

Thiết bị

Yêu cầu hoặc giải phóng thiết bị, đọc/ghi, định vị, lấy/đặt thuộc tính và điều khiển qua ioctl().

Nhiều hệ điều hành biểu diễn thiết bị như tệp để thống nhất cách lập trình I/O.



Lời gọi thông tin, giao tiếp và bảo vệ

Thông tin

Lấy thời gian, ngày, PID, thuộc tính hệ thống; ghi log và dữ liệu kế toán.

Giao tiếp

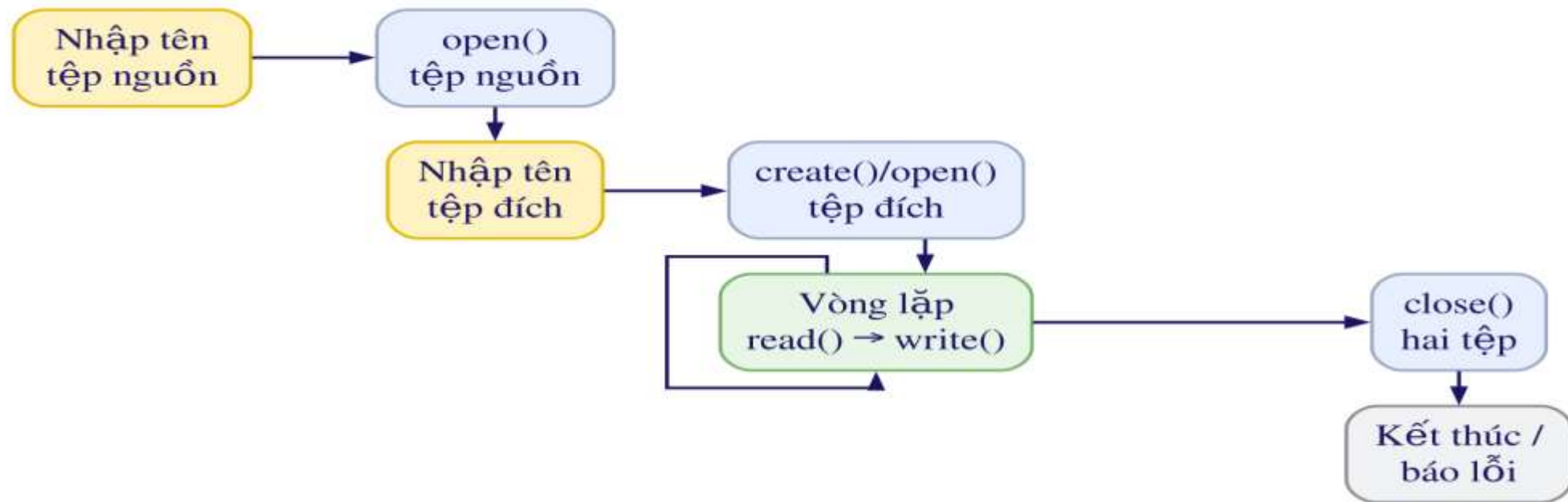
Tạo kết nối, gửi/nhận thông điệp, dùng pipe, socket hoặc vùng nhớ chia sẻ.

Bảo vệ

Lấy/đặt quyền, chủ sở hữu, danh tính người dùng và kiểm tra khả năng truy cập.

Các nhóm lời gọi phối hợp để chương trình quan sát hệ thống, trao đổi dữ liệu và hoạt động trong giới hạn quyền cho phép.

Ví dụ: chuỗi lời gọi hệ thống khi sao chép tệp



Chương trình phải kiểm tra mã trả về sau từng bước; nếu có lỗi, cần đóng tài nguyên đã mở và báo trạng thái phù hợp.



API chuẩn tạo lớp giao diện thuận tiện và khả chuyển

POSIX API

Chuẩn phổ biến trên UNIX/Linux;
ví dụ `open()`, `read()`, `write()`, `fork()`.

Win32 API

Giao diện của Windows; ví dụ
`CreateFile()`, `ReadFile()`,
`CreateProcess()`.

Java API

Lớp thư viện Java gọi dịch vụ của
JVM và hệ điều hành bên dưới.

Cùng một API có thể ánh xạ sang các system call khác nhau; chương trình thường không cần biết chi tiết cài đặt trong nhân.



2.5. Các chương trình hệ thống

Những tiện ích xây dựng môi trường thuận lợi để phát triển và thực thi chương trình

Chương trình hệ thống biến dịch vụ nhân thành công cụ sử dụng hằng ngày



Nhiều chương trình hệ thống là giao diện thuận tiện bao quanh một hoặc nhiều lời gọi hệ thống.



Nhân cung cấp cơ chế; chương trình hệ thống tổ chức cơ chế đó thành lệnh, công cụ và môi trường làm việc hoàn chỉnh.



Các nhóm chương trình hệ thống

Nhóm chức năng

- Quản lý tệp
- Thông tin trạng thái
- Soạn thảo tệp
- Hỗ trợ ngôn ngữ
- Nạp và thực thi
- Giao tiếp
- Dịch vụ nền

Ví dụ

cp / mv / rm	date / ps
editor	compiler
assembler	debugger
loader	shell
browser	daemon



Hai cách cài đặt trình thông dịch lệnh

Lệnh tích hợp trong shell

Mã thực hiện lệnh nằm ngay trong trình thông dịch. Gọi nhanh nhưng shell lớn và khó mở rộng khi có nhiều lệnh.

Lệnh là chương trình riêng

Shell tìm tệp thực thi, nạp và chạy với tham số. Dễ thêm lệnh mới mà không phải sửa trình thông dịch.

UNIX sử dụng mạnh cách thứ hai: rm, cp, ls... là các chương trình hệ thống độc lập được shell khởi chạy.



Chương trình hệ thống và chương trình ứng dụng

Chương trình hệ thống

Hỗ trợ vận hành, phát triển và khai thác hệ thống; thường cung cấp shell, tiện ích, thư viện, compiler, loader và daemon.

Chương trình ứng dụng

Giải quyết bài toán của người dùng như soạn thảo, kế toán, trình duyệt, trò chơi hoặc hệ quản trị cơ sở dữ liệu.

Ranh giới không tuyệt đối: cùng một chương trình có thể được xem là tiện ích hệ thống hoặc ứng dụng tùy mục đích và cách phân phối.



2.6. Cấu trúc hệ thống

Cách sắp xếp các thành phần và quy định giao diện tương tác bên trong hệ điều hành



Mục tiêu định hướng thiết kế hệ điều hành

Mục tiêu người dùng

Dễ dùng, dễ học, đáng tin cậy, an toàn, phản hồi tốt và phù hợp với môi trường ứng dụng.

Mục tiêu hệ thống

Dễ thiết kế, cài đặt, kiểm thử và bảo trì; hiệu quả, linh hoạt, kháng lỗi và dễ mở rộng.

Nguyên tắc quan trọng: Tách cơ chế khỏi chính sách và xác định giao diện rõ ràng giữa các thành phần.

Cấu trúc đơn giản và nhân đơn khối



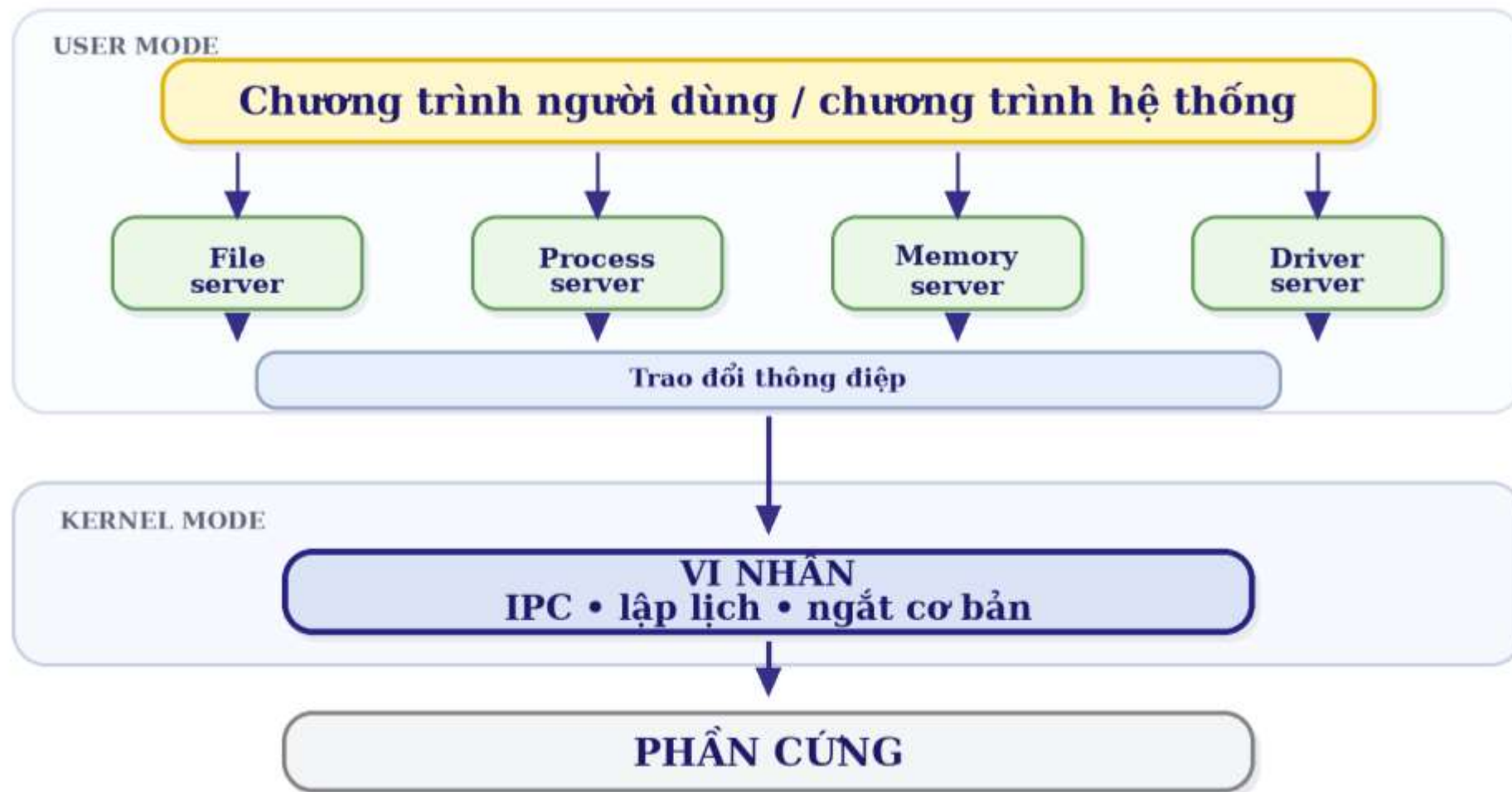
Cấu trúc UNIX: Chương trình hệ thống và nhân



Cấu trúc phân tầng



Cấu trúc vi nhân





Hai hướng tổ chức cấu trúc hệ điều hành

Tích hợp chặt

Đơn khối, mô-đun và cấu trúc lai ưu tiên hiệu năng, giao tiếp trực tiếp và khả năng kết hợp linh hoạt giữa các thành phần.

Tách biệt mạnh

Phân tầng và vi nhân tăng khả năng kiểm chứng, cách ly lỗi, bảo mật và mở rộng nhưng có thể phát sinh chi phí giao tiếp.

Không có cấu trúc tối ưu tuyệt đối: Lựa chọn phụ thuộc hiệu năng, độ tin cậy, phần cứng mục tiêu và chi phí phát triển/bảo trì.

Tổng kết Chương 2



Các thành phần hệ thống quản lý tiến trình, bộ nhớ, tệp, I/O, mạng và bảo vệ

Dịch vụ HĐH tạo môi trường thực thi thuận tiện và vận hành tài nguyên hiệu quả

Lời gọi hệ thống là giao diện lập trình có kiểm soát để truy cập dịch vụ của nhân

Chương trình hệ thống cung cấp shell, tiện ích, thư viện và công cụ phát triển

Cấu trúc đơn khối, phân tầng, vi nhân, mô-đun và lai thể hiện các đánh đổi khác nhau



Trao đổi và thảo luận

Vì sao ứng dụng thường gọi API thay vì gọi trực tiếp system call?

So sánh truyền tham số qua thanh ghi, khối nhớ và ngăn xếp.

Chương trình hệ thống khác chương trình ứng dụng ở vai trò nào?

Vì sao vi nhân tăng độ tin cậy nhưng có thể làm giảm hiệu năng?

Email: minhnl@utc.edu.vn